Pollis AI+

Pollis and AI, working together

New methods, with the answers already in hand.

An AI companion writes the method you are missing; Pollis has the reference values ready to test it.

AI is **off by default**, and Pollis works fully without it: every panel, notebook and workflow runs without AI. Pollis AI+ is for those who want AI beside them, and it builds on what makes Pollis trustworthy in the first place: results checked against published values.

The AI companion: creating new methods

available

Sometimes the method you need is not in Julia yet. Baltagi's panel-data textbook, for example, estimates Munnell's public capital data with the Wallace-Hussain and Amemiya random-effects estimators, which R has and Julia does not. A coding agent can write them in minutes; **checking** them is the slow part. Pollis makes the check part of the starting point.

1. **Name the method and its source:** the article, its data, and the tables of results it publishes.
2. **Pollis sets up the tests** before any code is written: every published value becomes a test.
3. **Your coding agent writes the method** and runs the tests until they pass.
4. **You see every number** beside the published one: full value, rounded, published, "same to 3 decimals". Never just "it matches".
5. **Keep it:** a tested Julia package with its vignette, ready to publish in the Pollis registry.

The nine methods on page 4 were made this way, with Claude Code, in **Compose** › **New Methods**.

Where the reference values come from

- **Classic papers and textbooks,** the same ones Pollis's workflows replicate.
- **Certified datasets** with reference results to many digits.
- **The same method in R and Python,** when no published value exists.

Why the tests come first

Rewriting a method in another language is no check: the same formulas give the same mistakes. Published values are written by others, checked by referees and used by many. With them in place, code written by an agent is held to the same standard as code written by an expert.

POLLIS AI+

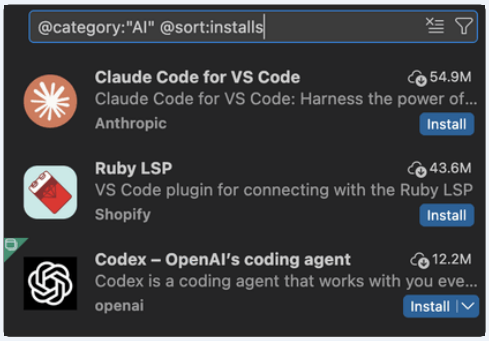
More ways to work with AI.

Your choice of tools, always optional.

AI extensions of your choice

available

Add the AI assistants you prefer: the most installed ones on Open VSX are one click away, in the Extensions view.



Pollis Agents: panels and toolboxes from a request

available

Ask for a panel, or a whole toolbox, in plain words. A coding agent of your choice, any command-line agent or a model running on your own computer, writes it in Pollis's panel format, checks it and packages it to share.

Your coding agent, in the cloud or on your computer

available

Choose Claude Code, Codex or OpenCode, with the provider's models or with a model running on your own computer through Ollama, LM Studio, llama.cpp or, on a Mac, Apple MLX. A local model costs nothing and your code never leaves your computer. Everything an agent does is marked with an Agent badge, since it costs tokens. **Model Routing** picks a lighter or a stronger model for each kind of task.

MCP Servers: official data for your agent

available

Compose › MCP Servers adds official data servers to your coding agent, starting with the Federal Reserve Bank of St. Louis's FRED: your agent searches the economic series and reads their notes and values while you talk. You sign in once, in the agent; Pollis holds no tokens.

Polish a report

available

Pollis turns a notebook into a LaTeX report or slides on its own, with no agent. Your coding agent can then polish the draft into a finished document, with sections, tables, captions and citations, using the numbers Pollis already computed: it writes the words, not the results.

Agents in Compose

planned

- **Correctness:** when Julia, R and Python disagree on a result, an agent looks for the reason.
- **Horse Races:** an agent speeds up a slow function and shows the timings before and after.
- **Collaboration:** invite your own agent to a session, from your computer. It joins as a named participant, sees only the chat and what is shared, answers in the chat and can share its own version. You pay its tokens, or nothing with a local model, and it does not count towards the five people. Each participant reads the chat in their own language, translated on their own computer, so nobody pays for the others' translation.
- **Voice Link:** talk to your own agent from your phone, on the road (page 3).
- **Practice:** an AI tutor gives hints and explains; teachers ask their agent for a first draft of new exercises, whose answers Pollis computes in Julia.

In one line

Pollis works without AI; with AI+, an agent writes the code and published results check it.

POLLIS AI+

Voice Link: your own agent, on the road.

Think aloud while you drive; your agent works on your computer.

Voice Link

planned

The best ideas often come away from the desk: driving to work, you see how to improve an analysis, or what to try next. With Voice Link you just say it. Your phone joins a Pollis session with your own computer, at home or in the office, and your coding agent there takes the idea further with Pollis while you keep driving. By the time you arrive, the work is done and waiting for you, and you have already heard what it found.

How it works

1. **You speak.** Your phone turns your words into text and sends them, enciphered, to your computer.
2. **Your agent works** with Pollis: it opens the panels, runs the workflows and the Julia code it needs.
3. **It writes it down:** a dated note of what it did and found, saved in your Pollis Workspace.
4. **You hear the answer:** a short summary that your phone reads aloud.
5. **You go on talking:** a follow-up, a new idea, or “leave it for when I arrive”.

Hands-free, eyes on the road

- **Voice in, voice out:** nothing to read and nothing to tap while you drive.
- **Your phone, paired once** with your computer by a QR code, and unpaired whenever you wish.
- **Enciphered** from your phone to your computer, as in every Pollis session.
- **The agent works only through Pollis:** panels, workflows and Julia; it does not run commands on your computer.
- **With a local model,** your work never leaves your computer and the answers cost nothing.

Why your own agent answers better than any chat

A chat app on your phone knows nothing about your work: it can only give a general answer. Your agent at home sits in the middle of it. In your Pollis Workspace it finds your analyses, the panels and workflows you chose, your own menus, and your own panels, workflows and reports, **structured and linked by you**. So it answers about *your* data and *your* question, and it builds on what you have already done instead of starting again. And its numbers are not a model’s guesses: Pollis computes them in Julia, checked as everywhere else. The agent writes the words, not the results.

One idea, three places

Voice Link is a Collaboration session, so others can join it: you on the road, your agent at home, and a colleague at their own computer, following the conversation, sharing code and adding ideas of their own. Their agent can join too. Pollis keeps it all in one place, ready for whoever picks it up next.

In one line

A chat answers anyone; with Voice Link, your own agent answers you, from your own work, with numbers Pollis computed.

POLLIS AI+

New methods for classic papers, made with AI.

Each one tested against the paper it was written for.

Pollis's workflows replicate classic papers. Where a paper used a method that Julia does not have, Claude Code wrote it, and the paper's own tables, or its authors' code, tested it. Each method is a Julia package in the Pollis registry, with a vignette that puts every published value beside Julia's: **Compose** › **New Methods** › **Browse Methods**.

Panel random effects

Baltagi, *Econometric Analysis of Panel Data*, Tables 2.1–3.7 and 9.4; Grunfeld, gasoline and public capital data.

Missing in Julia: the Wallace-Hussain, Amemiya, two-way Swamy-Arora and nested estimators, which R's plm has.

Result: 288 of 298 printed values to every digit. R's plm agrees to 7 significant digits.

Periodic GARCH

Bollerslev and Ghysels (1996), Table 2; daily exchange-rate returns.

Missing in Julia: GARCH with a calendar dummy in the variance, and the periodic model.

Result: 32 of 42 printed values to every digit, most of the rest in the last digit. Plain GARCH agrees with ARCHModels.jl to 12 decimals.

The Cox model with Breslow's ties

Dickson et al. (1989), Tables 2 and 4; the Mayo Clinic primary biliary cirrhosis data.

Missing in Julia: Breslow's rule for tied deaths, and predicted survival curves.

Result: the initial model and all seven years of predicted survival to every digit; 41 of 53 values in all. With Efron's ties, only 1 to 3 decimals.

Bootstrap and influence for GLMs

Dalal, Fowlkes and Hoadley (1989); the Challenger O-ring data.

Missing in Julia: the parametric bootstrap and delta-beta influence for binomial models.

Result: every published value, from the coefficients to the interval at 30 °F: (1.27, 5.95) for "about (1, 6)".

Out-of-bag random forests

Breiman (2001), Table 2 and Section 10; 19 data sets.

Missing in Julia: out-of-bag error and permutation importance of inputs.

Result: 34 of 38 forest error rates within simulation noise of the published ones.

Arcing classifiers (arc-fs)

Breiman (1998), the AdaBoost column; 20 data sets.

Missing in Julia: arc-fs, and CART trees pruned by cost complexity.

Result: 18 of 20 test errors within simulation noise of the published ones.

Propensity score matching

Dehejia and Wahba (1999), Tables 1 and 3, with the replies of Smith and Todd (2005) and Dehejia (2005); LaLonde's NSW data.

Missing in Julia: the propensity score methods of R's MatchIt, Matching and cobalt.

Result: 123 of 145 printed values to every digit, the other 22 in the last.

Threshold autoregression

Tong and Lim (1980), eqs. 9.1–9.3 and Figs. 8, 10 and 12; the Canadian lynx and sunspot data.

Missing in Julia: the SETAR model, its identification by AIC, and its limit cycles.

Result: 68 of 70 printed values to every digit, the other two in the last.

LSTNet

Lai et al. (2018), Table 2, and the authors' PyTorch code; Exchange-Rate data.

Missing in Julia: LSTNet-Skip as the authors' code has it.

Result: with the same weights, the authors' forecasts to 1.8×10^{-7}. Exchange-Rate at horizon 3: RSE 0.0256 and CORR 0.9706, against 0.0226 and 0.9735.

Where they disagree, you see it

Every vignette says, in *What the tests show* and *Still open*, which values do not agree, by how much, and why if known. One table entry looks like a misprint, with R's plm on Julia's side; one standard error is still unexplained. The tests can only be as good as the papers, and the gaps are part of the result.

© 2026 Antonio Saragga Seabra · Licensed under CC BY-SA 4.0 · Draft, 16 October 2026

contact@pollis.org